# Dissecting Linux and IoT Malware

Many reverse engineers working in antivirus companies spend most of their time analyzing 32-bit malware for Windows, and even the idea of analyzing something beyond that may be daunting at first. However, as we will see in this chapter, the ideas behind file formats and malware behavior have so many similarities that, once you become familiar with one of them, it will be easier and easier to analyze all subsequent ones.

In this chapter, we will mainly focus on malware for Linux and Unix-like systems. We will cover file formats that are used on these systems, go through various tools for static and dynamic analysis, including disassemblers, debuggers, and monitors, and explain the malware's behavior on Mirai.

By the end of this chapter, you will know how to start analyzing samples not only for the x86 architecture but also for various RISC platforms that are widely used in the **Internet of Things** (**IoT**) space.

To that end, this chapter is divided into the following sections:

- Explaining ELF files
- Common behavioral patterns
- Static and dynamic analysis of x86 (32- and 64-bit) samples
- Mirai, its clones, and more
- Static and dynamic analysis of RISC samples
- Handling other architectures

# Explaining ELF files

Many engineers think that ELF is the format for executable files only and is native to the Unix world from the start. The truth is that it was accepted as the default binary format for both Unix and Unix-like systems only around 20 years ago, in 1999. Another interesting point is that it is also used in shared libraries, core dumps, and object modules (that's why it is actually called an executable and linkable format). As a result, the common file extensions for ELF files include `.so`, `.ko`, `.o`, `.mod`, and others. It might also be a surprise for analysts who mainly work with Windows systems and got used to `.exe` files that one of the most common file extensions for ELF files is actually... none.

ELF files can also be found on multiple embedded systems and game consoles (for example, PlayStation and Wii), as well as mobile phones. Originally, Android used ELF libraries for the JNI, and now with the appearance of ART (Android Runtime), applications are being compiled/translated into ELF files as well.

# ELF structure

One of the main advantages of ELF that contributed to its popularity is that it is extremely flexible and supports multiple address sizes (32- and 64-bit), as well as endiannesses, which means it can work on many different architectures.

Here is a diagram describing a typical ELF structure:

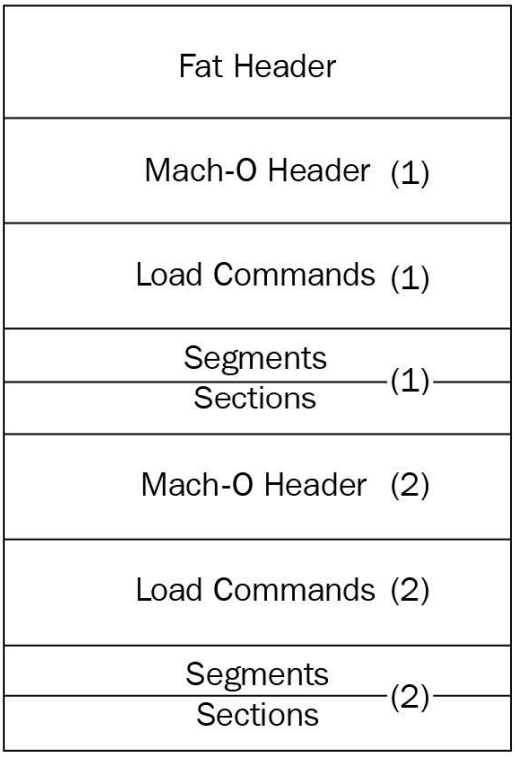


Figure 1: ELF structures for executable and linkable files

As we can see, it is slightly different for linkable and executable files, but in any case, it should start with a file header. It contains the 4-byte `\x7F'ELF'` signature at the beginning (part of the `e_ident` field, which will be described later), followed by several fields mainly specifying the file's format characteristics, some details of the target system, and information about other structure blocks. The size of this header can be either 52 or 64 bytes for 32- and 64-bit platforms, respectively (as for the 64-bit platforms, three of its fields are 8 bytes long in order to store 64-bit addresses, as opposed to the same three 4-byte fields for the 32-bit platforms).

Here are some of the fields that are useful for analysis:

- `e_ident`: This is a set of bytes responsible for ELF identification. For example, a 1-byte field at the offset 0x07 is supposed to define the target operating system (for example, `0x03` for Linux or `0x09` for FreeBSD), but it is commonly set to zero, so it can only give you a clue about the target OS in some cases.
- `e_type`: This 2-byte field at the offset 0x10 defines the type of the file—whether it is an executable, a shared object (`.so`), or maybe something else.
- `e_machine`: A 2-byte field at the offset 0x12, it is generally more useful as it specifies the target platform (instruction set), for example, 0x03 for x86 or 0x28 for ARM.
- `e_entry`: A 4- or 8-byte field (for the 32- or 64-bit platform, respectively) at the offset 0x18, this specifies the entry point of the sample. It points to the first instruction that will be executed once the process is created.

The file header is followed by the program header; its offset is stored in the `e_phoff` field. The main purpose of this block is to give the system enough information to load the file to memory when creating the process. For example, it contains fields describing the type of segment, its offset, virtual address, and size.

Finally, the section header contains information about each section, which includes its name, type, attributes, virtual address, offset, and size. Its offset is stored in the `e_shoff` field of the file header. From a reverse engineering perspective, it makes sense to pay attention to the code section (usually,this is `.text`), as well as the section containing strings (such as `.rodata`) as they can give plenty of information about malware's purposes.

There are many open source tools that can parse the ELF header and present it in a human-friendly way. Here are some of them:

- `readelf`
- `objdump`
- `elfdump`

# System calls

**System calls** (**syscalls**) is the interface between the program and the kernel of the OS it is running on. They allow user mode software to get access to things such as hardware-related or process management services in a structured and secure way.

Here are some examples of the system calls that are commonly used by malware.

# Filesystem

These syscalls provide all the necessary functionality to interact with the FS. Here are some examples:

- `open`/`openat`/`creat`: Open and possibly create a file
- `read`/`readv`/`preadv`: Get data from the file descriptor
- `write`/`writev`/`pwritev`: Put data in the file descriptor
- `readdir`/`getdents`: Read the content of the directory, for example, to search for files of interest
- `access`: Check file permissions, for example, for valuable data or own modules
- `chmod`: Change file permissions
- `chdir`/`chroot`: Change the current or root directory
- `rename`: Change the name of a file
- `unlink`/`unlinkat`: Can be used to delete a file, for example, to corrupt the system or hide traces of malware
- `rmdir`: Remove the directory

# Network

Network-related syscalls are built around sockets. So far, there are no syscalls working with high-level protocols such as HTTP. Here are the ones that are commonly used by malware:

- `socket`: Create a socket
- `connect`: Connect to the remote server, for example, a C&C or another malicious peer
- `bind`: Bind an address to the socket, for example, a port to listen on
- `listen`: Listen for connections on a particular socket

- `accept`: Accept a remote connection
- `send`/`sendto`/`write`/...: Send data, for example, to steal some information or request new commands
- `sendfile`: Move data between two descriptors. It is optimized in terms of performance compared to using the combination of `read` and `write`
- `recv`/`recvfrom`/`read`/...: Receive data, for example, new modules to deploy or new commands

# Process management

These syscalls can be used by malware to either create new processes or search for existing ones (for example, to detect AV software/reverse engineering tools or find a process containing valuable data). Here are some common examples:

- `fork`/`vfork`: Create a child process, for example, a copy of itself
- `execve`/`execveat`: Execute a specified program, for example, another module
- `prctl`: Allows various operations on the process, for example, a name change
- `kill`: Send a signal to the program, for example, to force it to stop operating

# Other

Some syscalls can be used by malware for more specific purposes, for example, self-defense:

- `signal`: This can be used to set a new handler for a particular signal and then invoke it to disrupt debugging, for example, for `SIGTRAP`, which is commonly used for breakpoints
- `ptrace`: This syscall is commonly used by debugging tools in order to trace executable files, but it can also be used by malware to detect their presence or to prevent them from doing it by tracing itself

Of course, there are many more syscalls, and the sample you're working on may use many of them in order to operate properly. The selection that's been provided describes some of the top picks that may be worth paying attention to when understanding malware functionality.

# Syscalls in assembly

When an engineer starts analyzing a sample and opens it in a disassembler, here is what syscalls will look like:

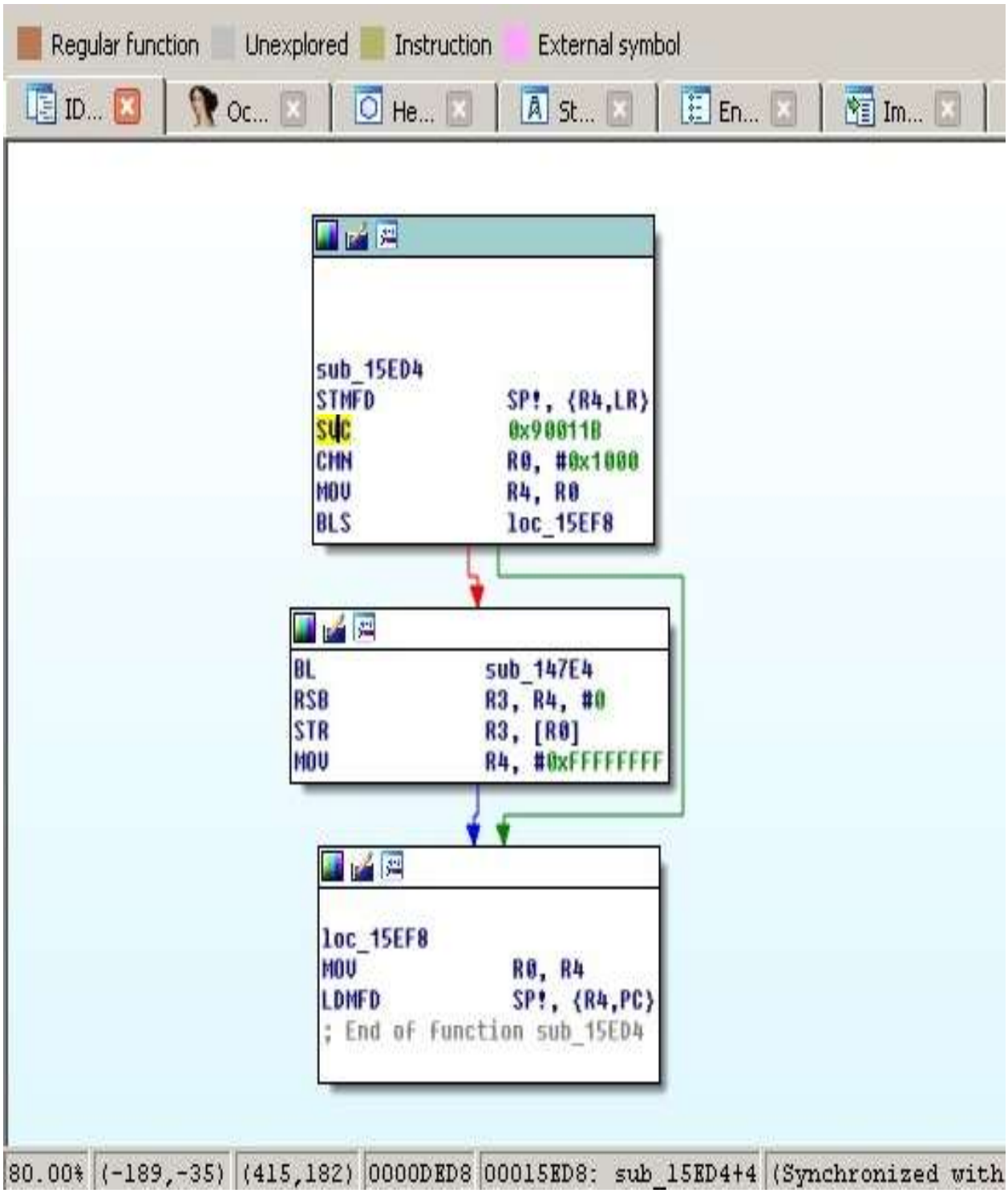

Figure 2: Mirai clone compiled for the ARM platform using the connect syscall

In the preceding screenshot, we can see that the number 0x90011B is used in assembly instead of a more human-friendly **connect** string. Hence, it is required to map these numbers to strings first. The exact approach will vary depending on

the tools that are used. For example, in IDA, in order to find the proper syscall mappings for ARM, the engineer needs to do the following:

1. First, they need to add the corresponding type library. Go to View | Open subviews | Type libraries (*Shift + F11* hotkey), then right-click | Load type library... (*Ins* hotkey) and choose gnulnx_arm (GNU C++ arm Linux).
2. Then, go to the Enums tab, right-click | Add enum... (*Ins* hotkey), choose Add standard enum by enum name, and add `MACRO_SYS`.
3. This enum will contain the list of all syscalls. It might be easier to present them in the hexademical format used in assembly rather than in the decimal format used by default. In order to do so, select this enum, then right-click | Edit enum (*Ctrl + E* hotkey) and choose the Hexademical representation instead of Decimal.
4. Now, it becomes easy to find the corresponding syscall, see the following figure:

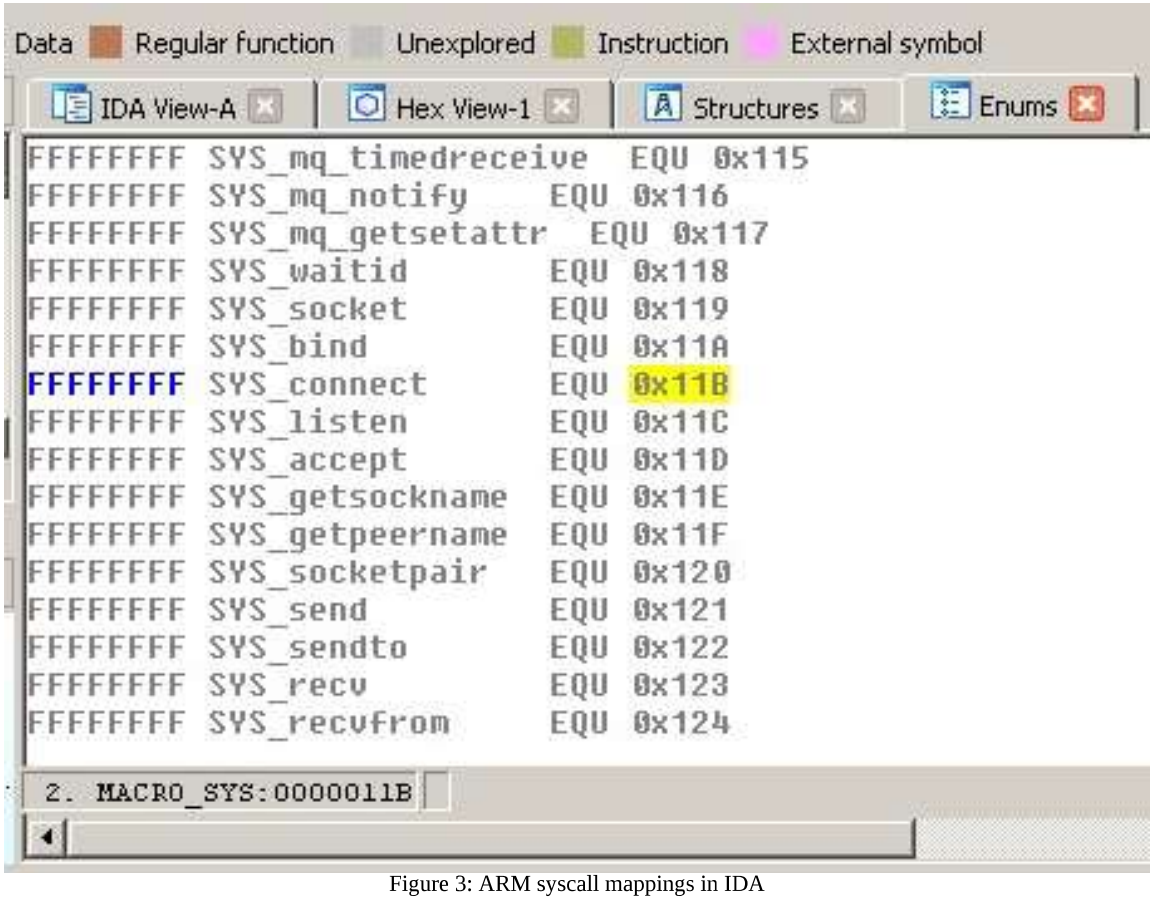

Figure 3: ARM syscall mappings in IDA

In this case, it definitely makes sense to use a script in order to find all the places where syscalls are being used throughout the code and map them to their actual

names to speed up the analysis.

# Common anti-reverse engineering tricks

Generic anti-reverse engineering tricks such as detecting breakpoints using checksums or exact match, stripping symbol information, incorporating data encryption, or using custom exception/signal handlers (setting them using the **signal** syscall we discussed previously) will work perfectly for ELF files pretty much the same way as for PE.

There are multiple ways the malware can take advantage of the ELF structure in order to complicate the analysis. The two most popular ways are as follows:

- **Make the sample unusual, but still follow the ELF specification**: In this case, malware complies with the documentation, but there are no compilers that would generate such code. An example of such a technique could be a wrong target OS specified in the header (we know that it can actually be `0`, which means this value is largely ignored by programs). Another example is a stripped section table, which is, as we saw earlier, actually optional for executable files.
- **Take advantage of the loose ELF header checks**: Here, malware uses an incorrect ELF structure, but as long as the software doesn't strictly follow the documentation when validating and loading it, it will still execute on the target system. An example can be incorrect information about sections.

In terms of syscalls, the most common way to detect debuggers and tools such as `strace` is to use `ptrace` with the `PTRACE_TRACEME` argument. The sample can also try to fork and then trace itself using this syscall in order to prevent debuggers from doing so. The `prctl` and `chroot` syscalls can be used to change the name of the process and change its root directory to avoid detection using these artefacts.

# Exploring common behavioral patterns

Generally, all malware of the same type share the same needs, regardless of the platform:

- It needs to get into the target system.
- In many cases, it needs to achieve persistence in order to survive the reboot.
- It may need to get a higher level of privileges, for example, to achieve the system-wide persistence or to get access to the valuable data.
- In many cases, it needs to communicate with the remote system (C&C) in order to do the following:
    1. Get commands
    2. Get new configuration
    3. Get self-updates, as well as additional payloads
    4. Upload responses, collected information, and files of interest
- Some malware families behave like worms, aiming to penetrate deeper into reached networks; this activity is commonly called a lateral movement.

The implementation depends on the target systems as they may use different default tools and file paths. In this section, we will go through common attack stages and provide examples of actual implementations.

# Initial delivery and lateral movement

There are multiple ways malware can get into the target system. While some approaches might be similar to the Windows platform, others will be different because of the different purposes they serve. Let's summarize the most common situations:

- **Default weak credentials**: Unfortunately, many companies manufacturing devices use very weak default credentials in order to remotely connect to the devices for maintenance purposes. While SSH and Telnet are the top choices of attackers in terms of the protocol being misused, other vectors are also possible, for example, web consoles. If we look at the list of hardcoded pairs found in the Mirai malware source code, we can see that somewhere around 60 combinations can give attackers access to several hundred thousand devices in a very short time. Here are some examples of them:
  - `root/12345`
  - `admin/1111`
  - `guest/guest`
  - `user/user`
  - `support/support`

- **Dynamic passwords**: Some companies tried to avoid this situation by using a so-called **password of the day**. However, the algorithm is generally easily accessible as it has to be implemented on the end user device, and it is too costly for the low-end devices to put it inside a dedicated chip or use a unique hardware ID as part of the secret. Eventually, it means that the infamous **security through obscurity** approach won't work in this case, and it becomes pretty straightforward for the attacker to generate the correct pairs of credentials every time they are needed.
- **Exploits**: Generally, the process of updating any system may require user interaction to go reliably, which is more troublesome for embedded devices compared to PCs. As a result, as long as some vulnerability becomes known, the list of devices it can affect remains huge over a long period of time. The same situation may happen to the generic Linux-based servers as well when the owners don't bother installing required updates as long as the

machine does its job. A good example of this is a TR-069 implementation being actively exploited by newer Mirai botnets.
- **Social engineering**: This approach is definitely not as popular as the others, but it still happens when the device owners are tricked into installing or executing some malicious code.

For lateral movement, often, the same approaches are being used. Apart from this, it is also possible to collect credentials on the first system and try to reuse them for nearby devices.

As we can see, there is no easy solution regarding how to fix these issues for already existing devices. Regarding the future, it will only happen when the device manufacturers become interested in bringing security to their devices (either because of customer demands, so it will become a competitive advantage, or because of the specific legislation imposed); it is quite unlikely that the situation will change drastically any time soon.

# Persistence

The persistence mechanisms can vary greatly, depending on the target system. In most cases, it relies on the automatic ways to execute code that are supported by the OS. Here are the most common examples of how this can be achieved:

- **Cron job**: This is probably the easiest cross-platform way to achieve persistence with the current level of privileges—that's why it was the first choice for developers of IoT malware. The idea here is that the attacker adds a new entry to `crontab`, which periodically attempts to execute (or download and execute) the payload. This approach guarantees the malware will be executed again after the reboot and, apart from this, it may revive malware if it is killed, either deliberately or accidentally. The easiest way to interact with `cron` is by using the `crontab` utility, but it is also possible to do this in `/var/spool/cron/crontabs/`. Another option is to modify `/etc/crontab` or place a script i `/etc/cron.d/` `or /etc/cron.hourly/ (.daily/.weekly/.monthly)` manually, but it will likely require elevated privileges.
- **Services**: There are many ways the services can be implemented, and all of these approaches require elevated privileges for malware to succeed:
  - `SysV-style init`: The most traditional approach that will work on a great range of systems. In this case, the payload (or a script calling it) needs to be placed to the `/etc/init.d/` location. After this, it can be invoked by using the symbolic link in the `/etc/rc?.d/` location. It is also possible to add malicious commands to the `/etc/inittab` file by defining runlevels directly. Another common option is to modify the `/etc/rc.local` file that's executed after normal system services.
  - `Upstart`: This is a younger service management package that was created by the former Canonical (creators of the Ubuntu OS) employee. Originally used in Ubuntu, it was later replaced by `systemd`. Chrome OS is another example of the system incorporating it. The main location of the configuration files in this case is `/etc/init`.
  - `systemd`: This system aims to replace `System V` and is now considered a de facto standard across multiple Linux distributions. The main location for the configuration files this time is `/etc/systemd`.
- **Profile configurations**: In this case, on bash, the current

user's `~/.bash_profile` (or `~/.bash_login` and the older `sh`'s `~/.profile` files) or `~/.bashrc` files are being misused with some malicious commands that were added there. The difference between these two is that the former is executed for login shells (that is, when the user logs in, either locally or remotely), while the latter is for interactive non-login shells (for example, when `/bin/bash` is being called, or a new Terminal window is opened). Interactive here means that it won't be executed if the bash just executes a shell script or is called with the `-c` argument. Other shells have their own profile files, for example, `zsh` uses the `.zprofile` file. This approach requires no elevated privileges. The `/etc/profile` file can be used in the same way but, in this case, elevated privileges are required as this file is shared across multiple users.

- **Desktop autostart**: Relatively rarely used by malware targeting IoT devices, which generally don't use graphics interfaces, this approach abuses autostart configurations for X desktops. The malicious `.desktop` files are placed in the `~/.config/autostart` location. Another related location for executing scripts this way is `~/.config/autostart-scripts`.
- **Actual file replacement**: This approach doesn't touch the configuration files and instead modifies/replaces actual original programs that are run periodically: either scripts or files. It will generally require elevated privileges in order to replace system files that are shared across multiple systems, but it can also be applied to some specific setup files with normal privileges.
- **SUID executables**: Another example, which is not commonly used by mass malware nowadays but is still possible, is to misuse SUID executables (files executed with the owner's privileges, for example, the ones belonging to the root user). For example, if the `find` utility has the SUID permission, it will allow the execution of virtually any command with escalated privileges using the `-exec` argument. Another common option is to modify the scripts that are executed by such files or change the environment variables they use so that they execute the attacker's script placed to some different location.

Other custom options, specific to certain operating systems, are also possible, but these are the most common cases often used by hackers and modern malware.

# Privilege escalation

As we can see, there are multiple ways malware can achieve persistence with the privileges it obtains immediately after penetration. It comes as no surprise that malware targeting IoT first of all focuses on them. For example, the VPNFilter malware incorporated `crontab` to achieve persistence; Torii (Mirai's clone) tries several techniques, one of which is using the local `~/.bashrc` file.

However, if at any stage the privilege escalation is required, there are several common ways of how this can be achieved:

- **Exploit**: Privilege escalation exploits are quite common, and there is always a chance that the owner of a particular system didn't patch it in time.
- **SUID executables**: As we discussed in the previous section, it is possible to execute commands with elevated privileges in the case of misconfigured SUID files.

- **Loose `sudo` permissions**: If the current user is allowed to execute any command using `sudo` without even a need to provide a password, it can be easily exploited by attackers. Even if the password is required, it can still be brute forced by the attackers.
- **Brute forcing credentials**: While this approach is likely not applicable to mass infection malware, it is possible to get access to the hash of the required password (for example, the one that belongs to the root) and then either brute-force it or use rainbow tables containing a huge amount of pre-computed pairs of passwords and their hashes in order to find a match.

There are other creative ways of how this can be achieved. For example, on older Linux kernels, it is possible to set the current directory of an attacker's program to `/etc/cron.d`, request the dump's creation in case of failure, and deliberately crash it. In this case, the dump, the content of which is controlled by the attacker, will be written to `/etc/cron.d` and then treated as a text file, and therefore its content would be executed with elevated privileges.